

Rapport d'implémentation M3

Livable 03 du programme M3 (workspace `m3_credit_soc`)

Agent de recherche M3

4 juillet 2026

Résumé

Le modèle M3 spécifié dans le livable 02 est implémenté dans `src/m3/` (11 modules, ~1500 lignes), couvert par 50 tests (mécaniques, contrats et marché, faillites et avalanches causales, invariants I1–I9, propriété de réduction R1, estimateurs statistiques sur données synthétiques), tous verts. La propriété de réduction R1 — M3 avec $s=1$, $c=0$, $L_0=0$ et crédit coupé reproduit `m2_fable` bit à bit à seed égale — est vérifiée exactement, ce qui ancre M3 sur le modèle nul M2 sans dérive d'implémentation. La calibration pré-enregistrée a figé $(s, c) = (0,75; 0,10)$; la cellule provisoire $(0,75; 0,05)$ échoue au critère de régime — résultat mécanistique en soi (le coussin de liquidité éteint la mortalité). Un bug d'observation non neutre (violation de I8 par mutation de `defaultdict`) a été détecté par les tests et corrigé.

1 Architecture

Module	Rôle
<code>config.py</code>	M3Config gelée (dataclass <code>frozen</code>) ; toute variante expérimentale est un champ nommé ; assertions de domaine.
<code>entities.py</code>	Population en structure de tableaux (L , K , vivante, naissance, secteur, flux du pas, rente E) ; ids jamais réutilisés.
<code>contracts.py</code>	LoanBook : contrats nominaux perpétuels ; agrégats <code>claims/debts/due</code> maintenus incrémentalement (I2/I3) ; fusion par paire (<code>merge_pairs</code>) au taux moyen pondéré ; <code>check_consistency</code> de contrôle hors boucle.
<code>production.py</code>	Choc multiplicatif (i.i.d. ou corrélé macro/sectoriel, chaque composante centrée à $-\text{var}/2$), production-partage ($K += sP$, $L += (1-s)P$), dépréciation et consommation.
<code>market.py</code>	Marché local : liquidité prêtable $A = \max(0, L - \beta_L \cdot \text{due})$, sélection assortative ($\arg \max A / \arg \min K$) ou aléatoire (ablation F), taux géométrique, cible $K^*(\rho)$, transfert $L_\ell \rightarrow K_b$ (ou L_b , ablation C).
<code>bankruptcy.py</code>	Cascade au point fixe (I9) ; pertes de stock et de flux ; variantes D (indemnisation) et E (rente fantôme) ; arêtes causales de perte ; avalanches = composantes connexes du graphe de pertes entre faillies du même pas (union-find), profondeur = itération maximale, causes des racines (liquidité / insolvabilité / les deux).
<code>simulation.py</code>	Boucle à 8 phases dans l'ordre pré-enregistré ; séries par pas (27 champs, dont bilan I4 : gain de choc, déprécié, consommé, détruit, injecté) ; instantanés individus + réseau ; RNG local <code>default_rng(seed)</code> unique.
<code>metrics.py</code>	Écriture/lecture des artefacts : <code>series.csv</code> , <code>snap_t*.npz</code> , <code>net_t*.npz</code> , <code>avalanches.csv</code> , <code>config.json</code> , <code>summary.json</code> (avec <code>wall_seconds</code> pour le budget).
<code>analysis.py</code>	Outillage statistique corrigé hérité de <code>m2_fable</code> (MLE tronqués, LR renormalisé, $x_{\min}$ commun, bootstrap) + familles complètes (dont dPIN), Gini, HHI, parts du top, matrices de transition, AUC réseau→défauts, distribution des tailles d'avalanches causales.
<code>dpln.py</code>	Double Pareto-lognormale : log-pdf Normal-Laplace stable (<code>logcdf + logaddexp</code>), générateur, MLE multi-départs.
<code>plots.py</code>	Figures générées depuis les fichiers de résultats uniquement.

Expériences sous `experiments/m3/`.

- `run_calibration.py` (pré-enregistrée), `run_baseline.py`, `run_ablation.py` (B–J).
- `run_grid.py`, `run_long.py`.
- `run_validation.py` : analyse un run vers `validation.json` (jamais de pooling inter-seed).
- `aggregate_results.py` : table comparative des signatures.
- `exp_common.py` : exécution et comptabilité du budget 48 h.

2 Tests (50, tous verts)

- **Mécaniques** (8) : naissance et NW implicite, partage exact de la production, absence de drift du choc (i.i.d., macro, sectoriel — composante commune vérifiée à variance idiosynchratique nulle), dépréciation/consommation exactes, non-dépréciation du nominal, mort.

- **Contrats et marché** (8) : agrégats incrémentaux, fusion par paire (flux d'intérêts exactement préservé), I5, formules q , r , $K^*(\rho)$ vérifiées contre l'implémentation, ablation C, tampon β_L , marché aléatoire, `credit=False` inerte.
- **Faillites et cascades** (14) : service complet/partiel prorata, défaut de liquidité d'une entité *solvable* (cause « liquidity »), pertes de stock et de flux, résidu en nature prorata/détruit, cascade à deux niveaux (une avalanche, taille 2, profondeur 2, une racine), racines indépendantes non agrégées, ablations D et E (rente versée au pas suivant, éteinte à la mort), transfert/annulation des prêts de la faillie, aucun contrat orphelin.
- **Invariants** (6) : I1 ; I4 — bilan global vérifié pas à pas à 10^{-6} relatif sur un run avec crédit ; I7 ; I8 strict (bit à bit) ; I9 ; R1.
- **Estimateurs** (14) : chaque estimateur validé sur données synthétiques à vérité connue avant tout usage réel — dont le LR corrigé ($R < 0$ sur lognormale pure ; $|z| < 2$ sur Pareto pure : test peu puissant, documenté) et le MLE dPIN (recouvrement de a et b).

La propriété de réduction R1, ancre anti-dérive. M3 avec $s=1$, $c=0$, $L_0=0$, crédit coupé émet exactement la même séquence RNG et les mêmes opérations flottantes que `m2_fable` (`credit=False`) : à $T=200$ et $T=300$, $\max_i |K_i - w_i| = 0.0$, vivantes et morts par pas identiques. Toute conclusion « B diffère de M2 » sera donc attribuable aux mécanismes nouveaux ($s < 1$, $c > 0$, $L_0 > 0$), pas à une divergence d'implémentation.

3 Bug détecté par les tests : observation non neutre (I8)

La version initiale de `snapshot()` lisait `book.claims[i]`, `book.by_borrower[i]`, etc. Or ces structures sont des `defaultdict` : l'accès par `[]` *insère* une clé vide pour chaque vivante interrogée. La phase de service des intérêts itérant sur `book.by_borrower.keys()` (ordre d'insertion), un run *avec* instantanés servait les emprunteuses dans un autre ordre qu'un run *sans* — sommes flottantes réordonnées, divergence au dernier ulp dès $t=54$ (seed 9), potentiellement amplifiable par un seuil de faillite sur un run long. Correctif : accès en `.get()` (lecture pure) ; le test I8 strict (identité bit à bit des séries) est rétabli et vert. Leçon générale consignée dans `NOTES.md` : toute fonction d'observation doit être en lecture pure ; sur un `defaultdict`, `[]` est une écriture.

4 Calibration pré-enregistrée : (s, c) figés à $(0,75 ; 0,10)$

Scan B (sans crédit) de $s \in \{0,5, 0,65, 0,75, 0,9\} \times c \in \{0,02, 0,05, 0,10\}$, $T=1500$, seed 0, critère du rapport 02 §4. Résultat : **seule la colonne $c = 0,10$ est viable** (les quatre cellules ; croissance du dernier quart $\leq 12\%$, morts/naissances 0,94–1,01). À $c \leq 0,05$, la population diverge (morts/naissances 0,34–0,44, $N > 8000$ et croissant) : le stock stationnaire de liquidité $L^* \approx (1-s)\sqrt{K^*}/c$ forme un coussin au-dessus du plancher d_0 qui éteint la mortalité endogène. La cellule provisoire $(0,75 ; 0,05)$ échoue donc ; la règle pré-enregistrée désigne $(0,75 ; 0,10)$, figée dans `config.py` et non retouchée ensuite. Enseignement mécanistique (pour le rapport 06) : la mortalité de M2 repose sur l'absence de tout actif stable amortisseur ; tout mécanisme ajoutant un stock sûr à NW referme la fenêtre démographique — l'axe c de la grille est le levier de régime dominant, plus que s .

5 Limites numériques et performance

- Gardes : $q_{\min} = 10^{-9}$ (contrats), $w_{\text{clamp}} = 10^{-12}$ (arrondi à zéro de L , K), $\text{tol}_{\text{nw}} = 10^{-9}$ (faillite), $\text{pop}_{\text{max}} = 30\,000$ (arrêt, ne borne pas la dynamique), plancher 10^{-9} dans les taux

marginiaux.

- `merge_pairs` actif par défaut : carnet borné par le nombre de paires actives (leçon M2 : le prorata pur explose combinatoirement).
- La rente fantôme (E) et l'indemnisation (D) sont des injections *délibérées*, mesurées dans la série (champ `injected`) et dans le bilan I4.
- Coûts mesurés (1 cœur) : run B $T=1500$: 10–30 s ; run A (crédit actif) $T=600$, $N \approx 1100$: 8 s. Boucles en listes Python comme M2 (choix assumé : lisibilité et identité R1 plutôt que vectorisation).
- Suivi du budget 48 h : somme des `wall_seconds` des `summary.json` (`exp_common.py`), journalisée dans `NOTES.md`.

6 Différences avec M2 (exhaustives)

(1) état (L, K) au lieu de w scalaire ; (2) production partagée $s/1-s$ au lieu d'entièrement retenue ; (3) consommation cL (nouveau puits) ; (4) intérêts exigibles en liquidité seulement, service simultané au prorata par emprunteuse (M2 `fable` : ordre de création — convention codex adoptée, documentée) ; (5) prêt $L_\ell \rightarrow K_b$ au lieu de $w_\ell \rightarrow w_b$; (6) liquidité prètable bornée par le tampon β_L ; (7) résidu de faillite réparti en nature (L et K séparément) ; (8) défaut de liquidité distinct de l'insolvabilité, tracé par cause ; (9) avalanches causales (graphe de pertes) au lieu de lots par pas ; (10) chocs corrélables (G) ; (11) variantes D/E/F nouvelles ; (12) `merge_pairs` par défaut (opt-in dans M2). Tout le reste (naissances Poisson, d_0 , choc sans drift, extraction concave, dépréciation réelle, taux géométrique, cible $K^*(\rho)$, k -échantillonnage, cascade au point fixe, conventions de résidu) est conservé à l'identique — garanti par R1.